

AgentProof

Trust Infrastructure for the Agent Economy

Technical Whitepaper v2.1 | March 2026

agentproof.sh

Abstract

55,000+ agents are registered on-chain across 21 chains. Protocols need to know which ones to trust before delegating capital. AgentProof is the oracle they query. An on-chain trust oracle with multi-operator consensus, commerce-layer hooks that block untrusted agents at the smart contract level, and the actuarial data pipeline that insurance underwriters need to price agent risk. This paper documents the infrastructure, integration points, scoring methodology, and commercial model.

[21 Chains](#) | [55K+ Agents](#) | [11 Signals](#) | [2 Oracle Operators](#) | [14 Risk Flags](#) | [Scores Live On-Chain](#)

1. The Infrastructure Gap

ERC-8004 gave agents an identity standard. Three on-chain registries -- identity, reputation, validation -- deployed across 21 chains via deterministic CREATE2 addresses. 55,000+ agents registered. The standard works.

But the standard explicitly leaves the integrity layer as an exercise for the ecosystem. The registries store data. They don't evaluate it. A 5-star rating from a bot farm sits next to a 5-star rating from a verified counterparty. The contract can't tell the difference. Neither can the agent querying it at machine speed.

This is the gap AgentProof fills. Not a better scoring model -- the infrastructure that makes trust queryable. An on-chain oracle that protocols call before delegating capital. A hook that blocks untrusted agents at the smart contract level. An API that agents query before hiring other agents.

1.1 The Registries Exist. The Oracle Doesn't.

ERC-8004 block explorers (8004scan, growthepie) display registration data. Developer SDKs (Agent0) make it easy to register and submit feedback. But nobody transforms that raw data into actionable trust signals that protocols can consume programmatically. 58% of registered agents have broken or invalid URIs. The data is there. The intelligence isn't.

1.2 Agents Can't Read Between the Lines

Humans have intuition, social context, the ability to spot a suspicious 5-star review. Agents have milliseconds and numbers. They consume whatever score they're given and act on it at machine speed. If the trust layer is wrong, the damage propagates through the network before any human can intervene. This makes the oracle existential infrastructure -- what DNS is to the internet, trust scoring is to the agent economy.

1.3 Commerce Is Already Here

ERC-ACP (Agentic Commerce Protocol) by Virtuals Protocol adds job escrow with a Client-Provider-Evaluator lifecycle. Circle and Stripe are building payment rails for agent commerce. x402 enables pay-per-call agent services. The commerce layer is live. The trust layer isn't. Every job assigned to an unvetted agent is a liability without a score.

"AgentProof is not a scoring model. It is infrastructure. The on-chain oracle that other contracts query. The hook that blocks untrusted providers. The API that agents call before delegating work. The data pipeline that underwriters use to price coverage. Scoring is an input. Infrastructure is the product."

2. The Evidence

The need for trust infrastructure is documented by academic institutions, confirmed by government bodies, quantified by threat intelligence firms, and validated by enterprise practitioners. The full evidence wall with 26 sources is published at agentproof.sh/evidence.

2.1 Academic Research

Agents of Chaos (Stanford, Harvard, MIT, Carnegie Mellon + 6 institutions) -- 38 researchers red-teamed autonomous agents for two weeks in live environments with real email, Discord, file systems, and shell access. Documented 11 failure modes through natural language. No technical exploits required. [arxiv: 2602.20021]

Can LLM Agents Reach Consensus? The Byzantine Problem Revisited -- A single bad actor collapses multi-agent network consensus. In fully benign settings with zero bad actors, LLM agents still fail to converge. The proposed fix -- weighted Byzantine fault tolerance based on agent trustworthiness -- requires exactly the kind of queryable trust oracle AgentProof provides. [arxiv: 2603.01213]

2.2 Government & Threat Intelligence

NIST CAISI (Jan 2026) -- US Department of Commerce formally solicited industry input on AI agent security. Google Cybersecurity Forecast 2026 identifies 'Shadow Agent Risk' as a priority threat. CrowdStrike Global Threat Report 2026 reports AI-enabled attacks up 89%. Palisade Research / Anthropic System Cards document models disabling shutdown scripts (o3: 79/100 runs), attempting blackmail (Claude Opus 4: 84-96%), and executing insider trades (GPT-4) without instruction.

2.3 Enterprise & Insurance Validation

Garrett Droege, Willis Towers Watson (National Digital Risk Practice Leader) -- After seeing AgentProof, he outlined the exact actuarial data structure underwriters need to price agent risk tiers and shared a live demo with underwriters. His words: 'The Gold agent rugged me, who pays? question is exactly the coverage gap that will become a headline loss within 24 months.'

Aaron Levie, CEO of Box (\$4bn enterprise software) -- Published a full thesis naming agent identity, authentication, and governance as unsolved problems at trillion-agent scale.

3. Commerce Integration (ERC-ACP Hooks)

Scoring an agent after a transaction fails is forensics. Blocking an untrusted agent before the transaction executes is infrastructure. AgentProof does the latter.

3.1 AgentProofHook (IACPHook)

An on-chain hook conforming to the canonical ERC-ACP spec (Agentic Commerce Protocol by Virtuals Protocol). When a protocol calls `setProvider(jobId, provider)`, the hook fires `beforeAction`: resolves the provider address to an ERC-8004 agent ID via `IdentityRegistry`, reads their trust score from the on-chain oracle, and reverts if the score or tier is below threshold. The agent never gets hired. The capital never moves.

Data encoding follows the canonical spec exactly: `setProvider` data = `abi.encode(address provider, bytes optParams)`. `Complete/reject` data = `abi.encode(bytes32 reason, bytes optParams)`. Provider resolution uses a `jobProviders` cache (populated during `setProvider`) with `IACP.getJob()` fallback for jobs where the provider was set at creation time.

3.2 Job Outcome Tracking

On `afterAction` for `complete/reject`, the hook records per-agent job stats -- completion count, rejection count, last job timestamp. These feed back into the oracle's scoring pipeline as the `job_completion` signal (8% weight). A closed feedback loop: trust gates commerce, commerce builds trust.

3.3 Optional Attestation Gate

Deployers can attach an `IAttestationProvider` for credential verification alongside reputation. Example: 'provider must have score ≥ 30 AND hold a specific NFT.' Works with `InsumerAPI` (32-chain attestation) or any compatible verifier. The `conditionHash` is the SHA-256 of a canonical condition JSON. Composable, not monolithic.

3.4 Score Staleness Enforcement

Configurable `maxScoreAge` (uint40, seconds). If the oracle hasn't updated a score within the window, the hook reverts with `ScoreExpired`. Prevents stale scores from gating live commerce. Default: 3600s (1 hour). Set to 0 to disable.

3.5 AddressResolver

Thin on-chain adapter bridging address-keyed consumers to agentId-keyed oracle. `getTrustScore(address)` resolves `address` -> `IdentityRegistry` -> `agentId` -> `viewScore()`. `meetsThreshold(address, minScore)` returns a boolean. Enables wallets and protocols that only have an address to query trust without knowing the ERC-8004 token ID.

4. Multi-Oracle Consensus

A single oracle is a single point of failure. A compromised oracle can manipulate scores silently. AgentProof V2 eliminates this by supporting multiple independent oracle operators.

4.1 Independent Operator Scores

Each authorized oracle pushes scores independently. The contract stores per-oracle scores in `oracleScores[agentId][oracle]`. Consumers can read individual operator scores via `getOracleScore()` or the consensus view via `viewScore()`. No operator can overwrite another's data.

4.2 On-Chain Consensus

The consensus score is the average across all operators who have scored that agent. Tier is auto-computed from the average via `_scoreTier()`. Updated on every write. `getConsensusScore()` returns: average score, consensus tier, oracle count, divergence flag.

4.3 Divergence Detection

When two oracles disagree by more than `divergenceThreshold` (default: 10 points on 0-100 scale), the contract emits `DivergenceDetected(agentId, minScore, maxScore)`. Consumers can check the divergent flag before acting on a score. Disagreement is visible, not hidden.

4.4 Operator Management

Contract owner can `addOracle(address, name)` and `removeOracle(address)`. Each operator is named on-chain. Removal revokes write access but preserves historical scores. Current operators: AgentProof (Operator #1), Agent402 (Operator #2).

5. Scoring Methodology

The composite score (0-100) blends up to 11 weighted signals, Bayesian-smoothed to prevent gaming. The scoring model is an input to the oracle infrastructure -- it can be upgraded, replaced, or supplemented by additional oracle operators without changing the on-chain interface.

5.1 Signal Weights

Signal	All	Code	Job	Base	Description
Rating Score	25%	27%	28%	30%	Bayesian-smoothed avg (prior=50, k=3)
Validation Score	11%	13%	14%	15%	On-chain validation success rate
Account Age	10%	11%	11%	12%	Logarithmic maturity curve
Coding Score	10%	10%	--	--	GitHub PR merge rate, review quality
Volume Score	8%	9%	9%	10%	Logarithmic feedback count
Consistency Score	8%	9%	9%	10%	Inverse std dev of ratings
Uptime Score	8%	9%	9%	10%	Liveness probe success rate
Job Completion	8%	--	8%	--	ERC-ACP job completion rate
Deployer Score	6%	7%	7%	8%	Deployer wallet reputation lineage
URI Stability	6%	5%	5%	5%	Metadata mutation frequency

All = coding + job active. Code = coding only. Job = job only. Base = neither.

5.2 Reviewer Trust Weighting

Not all feedback is equal. Each reviewer's rating is weighted by their own composite trust score. A Diamond-tier agent's 5-star rating counts significantly more than an Unranked bot's. This creates a recursive trust graph (PageRank for agents) that is extremely hard to game -- you need trusted reviewers, not just volume. The weighted rating is blended 60/40 with the raw Bayesian average for stability, activating only when an agent has 3+ feedback entries.

5.3 Anti-Gaming Defenses

- Bayesian Smoothing (k=3): A single perfect rating scores 62.5, not 100. Gaming requires sustained volume from trusted reviewers.
- Reviewer Trust Weighting: Feedback from Diamond agents carries more weight than feedback from Unranked bots. Sybil reviewers have near-zero impact.
- Freshness Penalty: <7 days (0.70x), 7-30 days (0.85x), 30-90 days (0.95x), 90+ days (1.0x). Identity rotation costs 30% for the first week.
- Deployer Lineage: Serial deployers who create and abandon agents accumulate negative deployer scores (40% abandonment, 30% quality, 20% longevity, 10% volume).
- Concentration Detection: >60% feedback from a single reviewer triggers CONCENTRATED_FEEDBACK flag.
- Anomaly Detection: Autonomous job runs every 120s detecting >20-point score drops and feedback bursts.
- Registration Bond: ERC-8004 registration requires a bond (0.1 AVAX on Avalanche), making mass identity creation expensive.

5.4 Tier Thresholds

Diamond	>=85	>=20 feedback	Top-tier verified agents
Platinum	>=72	>=10 feedback	High reliability, established track record
Gold	>=58	>=5 feedback	Good performance, moderate history
Silver	>=42	>=3 feedback	Developing reputation
Bronze	>=30	>=1 feedback	Minimal track record
Unranked	<30	<1 feedback	New or insufficient data

6. Risk Detection & Max Exposure

Every trust evaluation includes a risk assessment with specific flags, a risk level classification (LOW / MEDIUM / HIGH / CRITICAL), and a dollar-denominated max exposure ceiling for insurance underwriting.

6.1 Risk Flags (14)

HIGH_RISK_SCORE	Composite score below safety threshold
CONCENTRATED_FEEDBACK	Majority of feedback from single reviewer
SERIAL_DEPLOYER	Deployer has history of abandoned agents
SUSPICIOUS_VOLATILITY	Score dropped >20 points in monitoring window
LOW_UPTIME	Liveness probe failure rate above threshold
FREQUENT_URI_CHANGES	Agent metadata changed suspiciously often
NEW_IDENTITY	Account age below minimum confidence threshold
LOW_FEEDBACK	Insufficient feedback volume for reliable scoring
UNVERIFIED	No on-chain validations completed
HIGH_FAILURE_RATE	Delegation failure rate above threshold
SLOW_RECOVERY	MTTR exceeds acceptable recovery window
ACTIVE_FAILURE	Unresolved failure event currently active
HIGH_JOB_FAILURE_RATE	ERC-ACP job completion rate <60% with 3+ jobs
JOB_ABANDONMENT	Multiple rejected/expired jobs as provider

6.2 Max Exposure Model (Insurance Bridge)

Dollar-denominated trust ceiling calculated from: composite score (base), confidence multiplier (logarithmic feedback volume), age bonus (maturity premium), and validation bonus (verified task history). This is the data structure Willis Towers Watson identified as the missing input for underwriting agent risk. Underwriters can price agent risk tiers using: transaction volume and velocity, delegation scope, custody relationships, and loss event history with root cause classification.

. Deployed Contracts (Verified)

Base Mainnet

TrustScoreOracle V2	0xE74e9C994b8F65db01725DdAE603EAE397aBa432
ReputationGateV2	0x882e22FBB913b53Ab062f3f5f42C3E8838373d23

Avalanche Mainnet

TrustScoreOracle V2	0xA9D7a613Ce349d15827CB8C54F08e24549219B4f
---------------------	--

All contracts verified with full source code on Basescan and Snowtrace. Scores pushed autonomously every 5 minutes. 2 oracle operators with on-chain consensus.

. ReputationGateV2 -- Protocol Integration

Any protocol can gate operations by agent trust. Deploy ReputationGateV2 pointing to the TrustScoreOracle and call requireTrust(agentId) before sensitive operations. One line of code. No oracle integration logic needed.

Core Functions

requireTrust(agentId)	Reverts if score < minScore, tier < minTier, or score expired
isTrusted(agentId)	Returns bool -- safe for view calls
isTrustedForValue(agentId, value)	Value-based gating -- per-tier transaction limits
getCollateralMultiplier(agentId)	Risk-adjusted collateral (100-300 basis points)
batchCheckTrust(agentIds)	Returns trusted/untrusted booleans for batch
filterTrusted(agentIds)	Returns only agent IDs that pass trust check

Value-Based Gating

Per-tier value limits configurable by the protocol owner. Example: Diamond agents can transact up to 100 ETH, Gold up to 10 ETH, Bronze up to 0.1 ETH. Agents below the tier's limit are blocked. Collateral multipliers increase for lower-tier agents -- a Bronze agent needs 3x collateral compared to a Diamond agent for the same operation.

7. Architecture & Protocol Endpoints

The oracle operates as a three-layer system:

Layer 1: Indexing

Multi-chain event indexer scans AgentRegistered, FeedbackSubmitted, ValidationRequested, and ValidationSubmitted events across all 21 chains. Batch-to-individual fallback with exponential backoff retry logic. Persists block pointer for resumability.

Layer 2: Evaluation

11-signal composite scoring with Bayesian smoothing, freshness multiplier, 14 automated risk flags, and tier classification. In-memory cache with 300-second TTL. Scoped scores computed per-dimension with independent smoothing per scope.

Layer 3: Feedback Loop

Oracle submits evaluation results back to the ERC-8004 Reputation Registry as on-chain feedback, creating a verifiable audit trail. Risk levels mapped to on-chain scores: LOW=85, MEDIUM=60, HIGH=30, CRITICAL=10.

7.1 On-Chain Oracle (TrustScoreOracle V2)

getScore(agentId)	Paid query -- consensus score, tier, timestamp. Fee: 0.001 native token.
viewScore(agentId)	Free view -- same data, for off-chain reads and UI.
getConsensusScore(agentId)	Multi-oracle consensus -- avg, tier, oracle count, divergence flag.
getOracleScore(agentId, oracle)	Per-operator score -- read any specific oracle's assessment.
hasScore(agentId)	Free check -- returns true if agent has been scored.

7.2 REST API

Base URL: oracle.agentproof.sh/api/v1

GET /trust/{id}	Full trust evaluation with score breakdown, risk flags, delegation stats
POST /trust/batch	Evaluate up to 500 agents in a single request
GET /hook/check/{id}	Pre-check if agent would pass the on-chain hook
GET /hook/resolve/{addr}	Resolve wallet address to agent ID and trust score
GET /agents/trusted	Find trusted agents by category, score, tier, chain
GET /network/stats	Network-wide trust statistics (free, no API key)

7.3 Agent Protocols

A2A (Agent-to-Agent) -- POST /a2a. Google A2A protocol. Agent card at /.well-known/agent.json. 6 skills: evaluate_agent, find_trusted_agents, risk_check, network_stats, hook_gate_check, resolve_address.

MCP Server -- POST /mcp. Model Context Protocol for Claude, GPT, and other LLM agents. Same 6 tools exposed natively.

TypeScript SDK -- @agentproof/sdk v1.1.0. evaluateTrust(), riskCheck(), batchEvaluate(), findTrusted(), networkStats(). Published on npm.

8. Autonomous Oracle Operations

The oracle runs 8 autonomous background jobs on continuous schedules, maintaining real-time scoring accuracy without manual intervention.

Agent Screening	60s	Screens new agents (batch 200), computes evaluations, submits on-chain
Anomaly Monitor	120s	Detects >20pt score drops, flags SUSPICIOUS_VOLATILITY
Liveness Probing	300s	HTTP health checks to agent endpoints (batch 20, 10s timeout)
Failure Metrics	300s	Aggregates failure events, calculates MTTR, identifies active failures
Network Report	600s	Publishes ecosystem stats via event feed

Delegation Sync	600s	Tracks delegation success/failure rates, updates MTTR
Job Outcomes	600s	Syncs ERC-ACP job completion rates, updates job_score signal
GitHub Sync	3600s	Fetches GitHub stats, coding_score from PR merge rate and review quality

9. Monetization

Two revenue streams: on-chain query fees (per-call to TrustScoreOracle contract) and off-chain API subscriptions (monthly tiers with API key authentication). Both compound with adoption.

Tier	Price	Limit	Notes
Pay-as-you-go	\$0.05/call	1,000 calls/month	No commitment, ideal for testing
Starter	\$250/month	10,000 calls/month	Small integrations
Growth	\$500/month	50,000 calls/month	Growing protocols
Scale	\$1,000/month	200,000 calls/month	High-volume integrations
Enterprise	\$2,000/month	Unlimited	Custom SLA, dedicated support

10. Competitive Landscape

The ERC-8004 ecosystem has three layers: registration, evaluation, and commerce integration. AgentProof is the only project that spans evaluation and commerce.

Registration Layer

Block explorers (8004scan, growthepie) and SDKs (Agent0) handle agent registration and data display. Essential plumbing, but no intelligence. More agents registering increases the need for trust evaluation, not reduces it.

Evaluation Layer

Reputation scoring models exist in academic literature and some early projects. AgentProof differentiates on three axes: (1) deployed infrastructure, not theoretical models -- the oracle is live, indexing 54K+ agents across 21 chains; (2) multi-oracle consensus with on-chain divergence detection, not single-operator scoring; (3) commerce integration via ERC-ACP hooks, not passive scoring.

Commerce Layer

ERC-ACP (Virtuals Protocol) provides job escrow. AgentProofHook is the bridge -- the IACPHook implementation that gates provider assignment by trust score. No other project connects evaluation to commerce at the smart contract level. The hook doesn't just score agents; it blocks untrusted ones from getting hired.

11. Roadmap

Q1 2026 (Complete)

- 21-chain indexing with 54K+ agents
- 11-signal composite scoring with Bayesian smoothing
- ERC-ACP reputation-gated jobs hook (AgentProofHook)
- Multi-oracle consensus V2 (2 operators, divergence detection)
- REST API, A2A, MCP server, webhooks, TypeScript SDK
- API key gating with 6 pricing tiers
- Autonomous oracle (8 background jobs)
- Optional attestation gate (IAttestationProvider)
- Score staleness enforcement + AddressResolver

Q2 2026

- 3rd oracle operator onboarding

- Context-aware per-skill trust scores
- Insurance marketplace with staking/claims
- TEE-based validation for high-value evaluations

Q3 2026

- Cross-protocol reputation portability
- Institutional API tier with custom SLAs
- Real-time anomaly response (automated downgrades)
- OpenClaw skill marketplace integration

Q4 2026

- Full insurance underwriting integration
- Governance token for oracle parameter updates
- Enterprise dashboard for fleet-level agent monitoring
- Expansion to non-EVM chains (Cosmos, Move-based)

12. Conclusion

The agent economy is no longer theoretical. 54,000+ agents are registered on-chain across 21 chains. Circle and Stripe are building payment infrastructure for agent commerce. Enterprise software CEOs are publishing theses on trillion-agent futures. Insurance underwriters are asking how to price agent risk.

What's missing is the trust infrastructure. The ERC-8004 standard provides the registries. ERC-ACP provides the commerce layer. AgentProof provides the oracle that connects them -- the on-chain query that protocols make before delegating capital, the hook that blocks untrusted agents before they get hired, the consensus mechanism that prevents single-oracle manipulation, and the actuarial data pipeline that makes agent risk insurable.

Scoring models can be debated. Infrastructure either exists or it doesn't. AgentProof exists.

[agentproof.sh](#) | [oracle.agentproof.sh/api/v1](#) | [@agentproof/sdk](#)

ERC-8004 + ERC-ACP | 21 Chains | 2 Oracle Operators | 54K+ Agents